SYSTEM DESIGN

Microservices Architecture

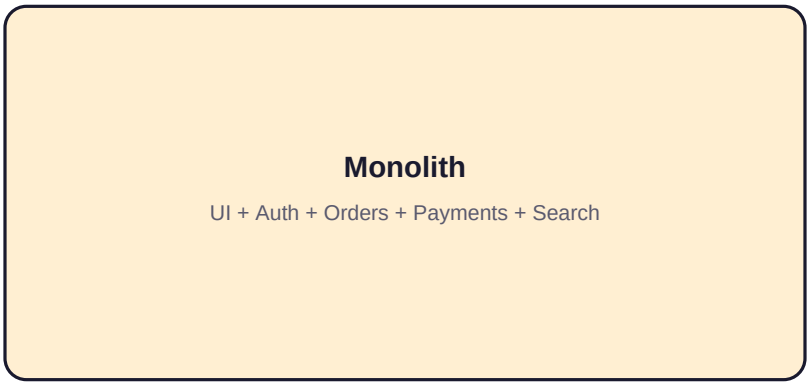
Decompose the monolith.
Independent services, independent scaling, independent deploys.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

The Monolith: One Deployable

All features in one codebase, one database, one release train.



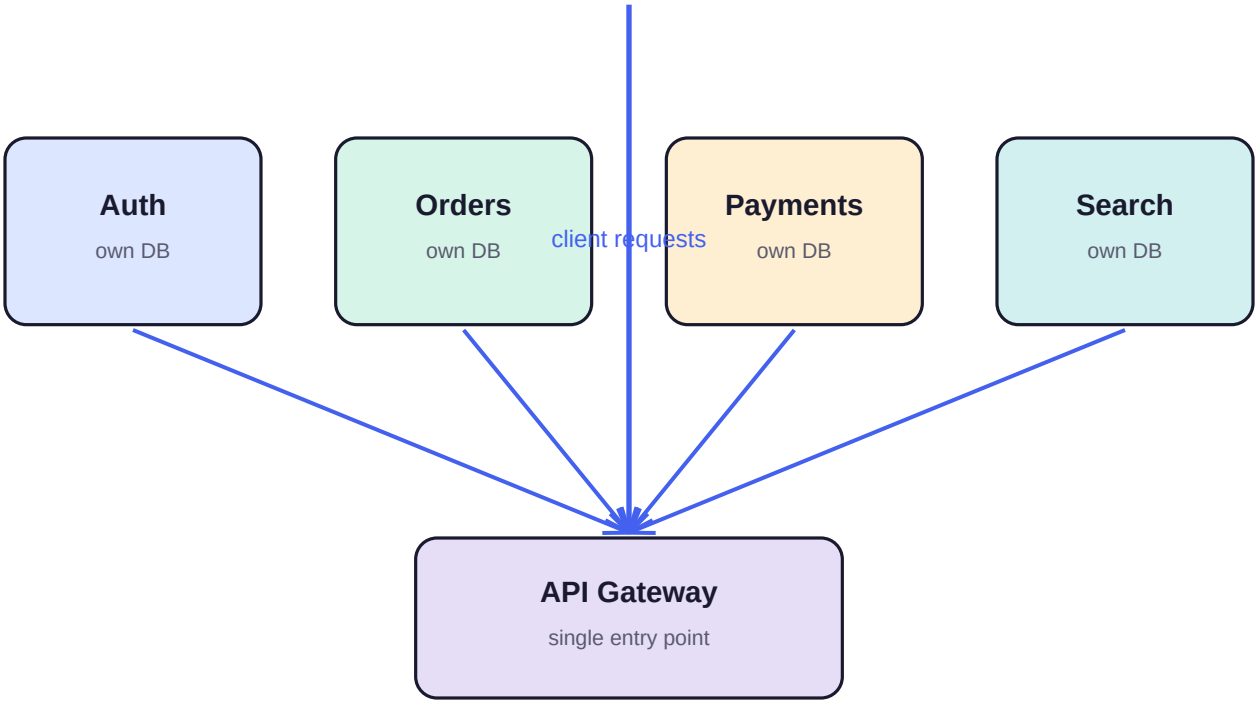
- Single codebase — simple to start
- One database — easy transactions
- One deploy — everything ships together
- Scales as a unit — wasteful

Great for small teams. Painful when teams and scale grow — every change risks everything.

STEP 2/6

Decompose by Business Capability

Split along domain boundaries — each service owns its data.



Each service deploys independently and scales independently. The gateway routes and composes.

STEP 3/6

Communication: Sync vs Async

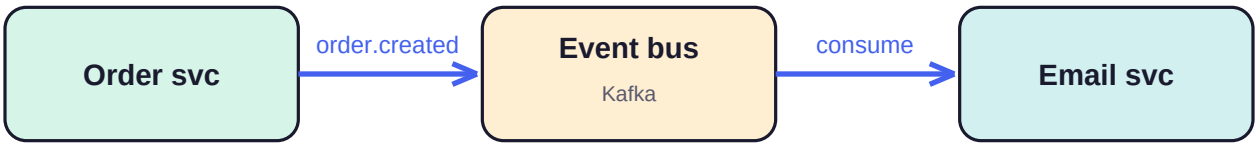
Services talk via REST/gRPC (sync) or events (async).

Synchronous
REST / gRPC

- Request → wait → response
- Simple, familiar
- Caller blocked; cascading failures

Asynchronous
Events / queues

- Publish event → move on
- Decoupled, resilient
- Eventual consistency



STEP 4/6

Service Discovery & Resilience

Find services dynamically; fail fast and gracefully.

Service Registry

Services register on boot; callers discover addresses (Consul, Eureka, k8s DNS).

Circuit Breaker

After N failures, stop calling for a cooldown — fail fast instead of piling up.

Retries + Backoff

Retry transient failures with exponential backoff + jitter.

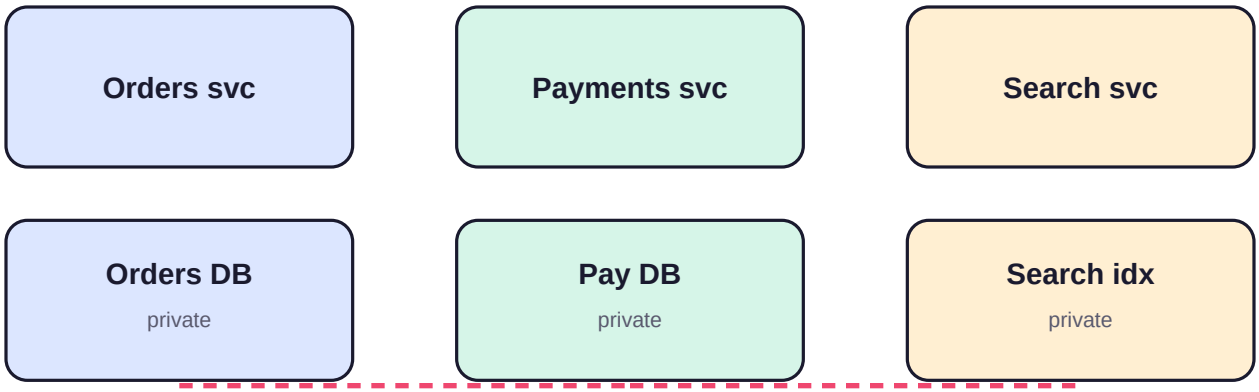
Bulkheads

Isolate resources per dependency so one slow service can't sink the ship.

STEP 5/6

Data: Database per Service

No shared databases — each service owns its schema.



✗ NO direct cross-service DB access

Share via APIs and events — never via the database.

The saga pattern coordinates multi-service transactions with compensating actions instead of 2PC.

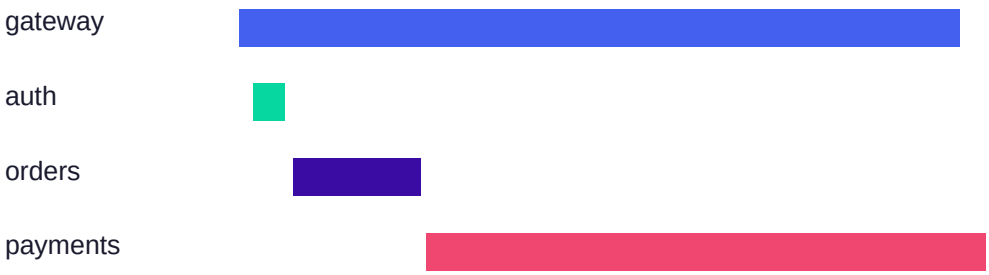
STEP 6/6

Observability: See the Whole System

Distributed tracing, metrics, and logs — correlated by request ID.

Trace: req-abc123

gateway → auth (12ms) → orders (48ms) → payments (210ms ■)



One slow span (payments) explains the whole slow request. Metrics + logs + traces = the three pillars.

STEP ✓/✓

Key Takeaways

Microservices — Remember

- Decompose by business capability; each service owns its code, data, and deploys.
- Communicate via sync APIs (simple) or async events (resilient) — choose per interaction.
- Resilience: service discovery, circuit breakers, retries with backoff, bulkheads.
- Database-per-service; coordinate with sagas, never shared tables.
- Observability is mandatory: distributed tracing with one request ID across services.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev